# inverse_development_7d

October 22, 2026

## 1 7D Inverse Development

A&S stands for "Abdulkhaev and Shirokov" the authors of a 7D multivector inverse.
I became aware of this algorithm from view graphs they presented at ENGAGE 2025. This algorithm is described here: Kamron Abdulkhaev at ENGAGE 2025 This notebook begins with code constructed from those viewgraphs. The algorithm is verified to work in Cl(7) using a home-grown Python package `clif` being used here.

Noticing that the algorithm contains a determinant formula attributed to Acus & Dargys - Inverse of Multivector: Beyond P+Q=5 Threshold, I substituted a different determinant version from that paper – one that held the benefit of simplification in a special way. I produced an *involution version* using this other determinant formula, and verified it in Cl(7).

With little modification, I was able to substitute my *tweak* formulas, developed for another 6D inverse, into this 7D program. This third and last version of a 7D inverse is the ultimate purpose here.

My work using tweak formulas is just an algebraic simplification and optimization of these algorthms. The main attritution to Kamron Abdulkhaev and Dmitry Shirokov (A&S) is the clever use of involutions which keeps the implementation 7D. The idea of using a 6D algorithm to solve the 7D problem should be attrituted to Eckhard Hitzer and Steven Sangwine (H&S). The variety of determinant formulas making any of these algorithms possible is attributed to A. Acus and A. Dargys (A&D).

[1]:
```
#=== This cell is for the construction of a 7D -> 6D grade table ===#
# Once the printed result of M[][] here,
# the next cell defines MT()() in like-fashion.

def comp(x,d=7): return ((1<<d)-1)^x
def vcomp(V): return [comp(v) for v in V]
# 7D vector basis -- A&S call "generators"
M = [[0,comp(0)]]
e = [1,1<<1,1<<2,1<<3,1<<4,1<<5,1<<6]
print( 'e = ', [f'{a:07b}' for a in e] )
# 6D bivector basis - e[0]=e1 is chosen for the "common vector",
# e[6]=e7 would have been my choice (turns out not to matter)
b = [ e[0] + a for a in e[1:] ]
print( 'b = ', [f'{a:07b}' for a in b] )
M += [ b + vcomp(b) ]
```

```
b2 = []
for i in range(len(b)-1):
    for j in range(i+1,len(b)):
        b2 += [b[i]^b[j]]
M += [ b2 + vcomp(b2) ]
b3 = []
for i in range(len(b)-2):
    for j in range(i+1,len(b)-1):
        for k in range(j+1,len(b)):
            b3 += [b[i]^b[j]^b[k]]
M += [ b3 + vcomp(b3) ]
b4 = []
for i in range(len(b)-3):
    for j in range(i+1,len(b)-2):
        for k in range(j+1,len(b)-1):
            for l in range(k+1,len(b)):
                b4 += [b[i]^b[j]^b[k]^b[l]]
M += [ b4 + vcomp(b4) ]
b5 = e[1:]
M += [ b5 + vcomp(b5) ]
M += [ [e[0], comp(e[0])] ]

# use the ordinals in M for computations
# print the elements each bin -- reproduce appendix 2
def label( n ):
    s = 'e'
    for i in range(1,8):
        if 0<((1<<(i-1))&n):
            s += str(i)
    return s

print( '\nA 6D Grading of 7D' )
for G in range(7):
    s = str(G) + ' = {'
    for m in M[G]:
        s += label( m ) + ','
    print( s[:-1] + '}' )

print( '\nA Graded Response to 7D' )
for m in M:
    print( m )
```

```
e =  ['0000001', '0000010', '0000100', '0001000', '0010000', '0100000',
'1000000']
b =  ['0000011', '0000101', '0001001', '0010001', '0100001', '1000001']

A 6D Grading of 7D
0 = {e,e1234567}
```

```
1 = {e12,e13,e14,e15,e16,e17,e34567,e24567,e23567,e23467,e23457,e23456}
2 = {e23,e24,e25,e26,e27,e34,e35,e36,e37,e45,e46,e47,e56,e57,e67,e14567,e13567,e
13467,e13457,e13456,e12567,e12467,e12457,e12456,e12367,e12357,e12356,e12347,e123
46,e12345}
3 = {e1234,e1235,e1236,e1237,e1245,e1246,e1247,e1256,e1257,e1267,e1345,e1346,e13
47,e1356,e1357,e1367,e1456,e1457,e1467,e1567,e567,e467,e457,e456,e367,e357,e356,
e347,e346,e345,e267,e257,e256,e247,e246,e245,e237,e236,e235,e234}
4 = {e2345,e2346,e2347,e2356,e2357,e2367,e2456,e2457,e2467,e2567,e3456,e3457,e34
67,e3567,e4567,e167,e157,e156,e147,e146,e145,e137,e136,e135,e134,e127,e126,e125,
e124,e123}
5 = {e2,e3,e4,e5,e6,e7,e134567,e124567,e123567,e123467,e123457,e123456}
6 = {e1,e234567}

A Graded Response to 7D
[0, 127]
[3, 5, 9, 17, 33, 65, 124, 122, 118, 110, 94, 62]
[6, 10, 18, 34, 66, 12, 20, 36, 68, 24, 40, 72, 48, 80, 96, 121, 117, 109, 93,
61, 115, 107, 91, 59, 103, 87, 55, 79, 47, 31]
[15, 23, 39, 71, 27, 43, 75, 51, 83, 99, 29, 45, 77, 53, 85, 101, 57, 89, 105,
113, 112, 104, 88, 56, 100, 84, 52, 76, 44, 28, 98, 82, 50, 74, 42, 26, 70, 38,
22, 14]
[30, 46, 78, 54, 86, 102, 58, 90, 106, 114, 60, 92, 108, 116, 120, 97, 81, 49,
73, 41, 25, 69, 37, 21, 13, 67, 35, 19, 11, 7]
[2, 4, 8, 16, 32, 64, 125, 123, 119, 111, 95, 63]
[1, 126]
```

[2]:
```
print( [len(m) for m in M] )
print( "compare with row 6 of Pascal's triangle" )
B = [1]
print(B)
Pascal = [B]
for j in range(8):
    B = [ (B+[0])[i]+([0]+B)[i] for i in range(len(B)+1) ]
    print(B)
    Pascal += [B]
```

```
[2, 12, 30, 40, 30, 12, 2]
compare with row 6 of Pascal's triangle
[1]
[1, 1]
[1, 2, 1]
[1, 3, 3, 1]
[1, 4, 6, 4, 1]
[1, 5, 10, 10, 5, 1]
[1, 6, 15, 20, 15, 6, 1]
[1, 7, 21, 35, 35, 21, 7, 1]
[1, 8, 28, 56, 70, 56, 28, 8, 1]
```

[3]:
```
dimensions = 7
def bases():
    return 1<<dimensions

# The grade of an element is so useful, we table it here
Grade = [0]
while len(Grade) < bases():
    Grade += [g+1 for g in Grade]

MT = (
    (0, 127),
    (3, 5, 9, 17, 33, 65, 124, 122, 118, 110, 94, 62),
    (6, 10, 18, 34, 66, 12, 20, 36, 68, 24, 40, 72, 48, 80, 96, 121,
     117, 109, 93, 61, 115, 107, 91, 59, 103, 87, 55, 79, 47, 31),
    (15, 23, 39, 71, 27, 43, 75, 51, 83, 99, 29, 45, 77, 53, 85, 101,
     57, 89, 105, 113, 112, 104, 88, 56, 100, 84, 52, 76, 44, 28, 98,
     82, 50, 74, 42, 26, 70, 38, 22, 14),
    (30, 46, 78, 54, 86, 102, 58, 90, 106, 114, 60, 92, 108, 116, 120,
     97, 81, 49, 73, 41, 25, 69, 37, 21, 13, 67, 35, 19, 11, 7),
    (2, 4, 8, 16, 32, 64, 125, 123, 119, 111, 95, 63),
    (1, 126)
)

def abdul_signs( V ):
    signs = [1]*128
    for v in V:
        for n in MT[v]:
            signs[n] = -1
    return signs

Signs4   = abdul_signs( [4] )
Signs236 = abdul_signs( [2,3,6] )
Signs145 = abdul_signs( [1,4,5] )
GISigns = [ [1,-1][1 & g] for g in Grade ] # for "grade involution"

#print( Signs4 )
print( "Signs236 = ", Signs236 )
#print( Signs145 )
#print( GISigns )

def abdulkhaev_7d_inverse( U ):
    Up = Signs236 * U
    W = U * Up
    Wc = W * (Signs145 * (W * W))
    Wa = Signs4 * W
    Wac = Wa * (Signs145 * (Wa * Wa))
    Waca = Signs4 * Wac
```

```
    y = Up * ((1/3)*Wc + (2/3)*Waca)
    Uy = U * y
    gUy = GISigns * Uy # grade involution of Uy
    DetU = Uy * gUy
    I = (1/DetU.Reg[0]) * y * gUy
    return I

import clif.context as Clif
from clif.multivector import Accum
import clif.util as util
Clif.Cl(7)
U = Accum.random()
I = abdulkhaev_7d_inverse( U )
print('\nProve inverse: 1/U * U = 1?')
util.print_sparse( U * I )
```

```
Signs236 =  [1, -1, 1, 1, 1, 1, -1, 1, 1, 1, -1, 1, -1, 1, -1, -1, 1, 1, -1, 1,
-1, 1, -1, -1, -1, 1, -1, -1, -1, -1, 1, -1, 1, 1, -1, 1, -1, 1, -1, -1, -1, 1,
-1, -1, -1, -1, 1, -1, -1, 1, -1, -1, -1, -1, 1, -1, -1, -1, 1, -1, 1, -1, 1, 1,
1, 1, -1, 1, -1, 1, -1, -1, -1, 1, -1, -1, -1, -1, 1, -1, -1, 1, -1, -1, -1, -1,
1, -1, -1, -1, 1, -1, 1, -1, 1, 1, -1, 1, -1, -1, -1, -1, 1, -1, -1, -1, 1, -1,
1, -1, 1, 1, -1, -1, 1, -1, 1, -1, 1, 1, 1, -1, 1, 1, 1, 1, -1, 1]

Prove inverse: 1/U * U = 1?
  0. 00000000      1.00000000
```

The construction above confirms for me that what has been accomplished is a clever generalization of involutions that replaces the Hitzer-Sangwine algorithm which uses a 6D inverse to perform a 7D inverse. What is going on with the Ws looks like one of the Acus-Dargys determinants. In fact, it is the first one A&D selected for their table 3 summarizing inverse formulas over dimensions.

$$\frac{1}{3}Wb_{145}(WW) + \frac{2}{3}b_4(b_4(W)b_{145}(b_4(W)b_4(W))$$

where $b_{145}(W)$ is a particular involution on $W$ which negates elements listed in the table under 1, 4, and 5. The structure of that table reflects an embedding of 7D in 6D.

Now, consider that there is an equivalent formula which only uses the 145 involution.

$$\frac{1}{3}Wb_{145}(WW) + \frac{2}{3}b_{145}(b_{145}(W)b_{145}(b_{145}(W)b_{145}(W))$$

I make use of the fact that $b_{145}$ is the same as $b_0$ in the A&D environment to good benefit. But, we should look closer how the involutions have changed for 7D. I verified that the $MT[2,3,6]$ elements of $W$ are indeed zero; so treating $W$ like a *self-reverse* multivector seems appropriate in this context. So we can write $ b_{145}(W) = - b_0(W) $ and compute

$$-\frac{1}{3}Wb_0(WW) - \frac{2}{3}b_0(b_0(W)b_0(b_0(W)b_0(W))$$

in good conscience. The involutions in this formula are much simpler.

Note that there is a point in the algorithm where we have a $V = s + p * e1234567$. At this point, we need to multiply by $V' = s - p * e1234567$ to get to a scalar. $V'V = s^2 - p^2(e1234567)^2$ is a scalar alright, but we can't assume the value of $(e1234567)^2$.

Let's consider if the form of the involution which uses e7 instead of e1 has any benefit or offers any insight.

First we distribute the *even space* in 7D over a 6D bivector grading.
We choose a special vector, e7 here instead of e1. - 0. (1) scalar - 1. (6) bivectors containing e7 - 2. (15) the other bivectors (that is: the bivectors in 6D) - 3. (20) quadravectors containing e7 - 4. (15) the other quadravectors (that is: the quadravectors in 6D) - 5. (6) hexavectors containing e7 - 6. (1) the 6D pseudoscalar The *odd space* completes the design, where we include the 7D complement at each "grade".

Of course the involution that we want is the one which imitates reversion choosing "grades" {2,3,6}. At this point in our algorithm conversion, this involution is the only one left.

The advantage of this scheme is that our base grading in the even grades is just the underlying 6D even elements (which of course are still even in 7D). The base grading in the odd grades is seen to be just the 6D odd elements made even by the the inclusion of `e7`.

[4]:
```
# alternate involution
table = []
for n in range(7):
    table += [[]]
for n in range(1<<7):
    g = Grade[n]
    if g in [0,2,4,6]:
        if 0 == 0b100_0000 & n:
            table[g] += [n]
        else:
            table[g-1] += [n]

alt236 = [1]*128
elements = table[2]+table[3]+table[6]
for e in elements:
    alt236[e] *= -1
    alt236[0b111_1111 ^ e] *= -1

print( '\nAn Alternate 6D Grading of 7D' )
for n in range(7):
    s = str(n) + ' = {'
    for m in table[n]:
        s += label( m ) + ','
    for m in table[n]:
        s += label( 0b111_1111 ^ m ) + ','
    print( s[:-1] + '}' )
```

```
An Alternate 6D Grading of 7D
```

```
0 = {e,e1234567}
1 = {e17,e27,e37,e47,e57,e67,e23456,e13456,e12456,e12356,e12346,e12345}
2 = {e12,e13,e23,e14,e24,e34,e15,e25,e35,e45,e16,e26,e36,e46,e56,e34567,e24567,e
14567,e23567,e13567,e12567,e23467,e13467,e12467,e12367,e23457,e13457,e12457,e123
57,e12347}
3 = {e1237,e1247,e1347,e2347,e1257,e1357,e2357,e1457,e2457,e3457,e1267,e1367,e23
67,e1467,e2467,e3467,e1567,e2567,e3567,e4567,e456,e356,e256,e156,e346,e246,e146,
e236,e136,e126,e345,e245,e145,e235,e135,e125,e234,e134,e124,e123}
4 = {e1234,e1235,e1245,e1345,e2345,e1236,e1246,e1346,e2346,e1256,e1356,e2356,e14
56,e2456,e3456,e567,e467,e367,e267,e167,e457,e357,e257,e157,e347,e247,e147,e237,
e137,e127}
5 = {e123457,e123467,e123567,e124567,e134567,e234567,e6,e5,e4,e3,e2,e1}
6 = {e123456,e7}
```

Before we leave alt236 as an involution, let's give it a serious description. First we need a description of the grading:

| | | |
|---|---|---|
| 0. | scalar | 6D pseudoscalar & e7 |
| 1. | 6D vectors & e7 | 6D 5-vectors |
| 2. | 6D 2-vectors | 6D 4-vectors & e7 |
| 3. | 6D 3-vectors & e7 | 6D 3-vectors |
| 4. | 6D 4-vectors | 6D 2-vectors & e7 |
| 5. | 6D 5-vectors & e7 | 6D vectors |
| 6. | 6D pseudoscalar | e7 |

So, the elements we negate for the alt236 involution is:

| | | |
|---|---|---|
| 2. | 6D 2-vectors | 6D 4-vectors & e7 |
| 3. | 6D 3-vectors & e7 | 6D 3-vectors |
| 6. | 6D pseudoscalar | e7 |

Or, considering the first half is 6D, we can describe alt236 as: grades 2,3,6 from the first half grades 1,4,5 from the second half

Yet another way to describe involution by direct product is: modify the standard 7D reversion involution direct-product by negating the 2nd half.

[5]:
```
Talt = [[1,1,-1,-1][3&Grade[i]] for i in range(1<<7)]
for i in range(1<<6,len(Talt)): Talt[i] *= -1
for i in range(len(Talt)):
    print(f"{i:07b} {alt236[i]} {Talt[i]}")
```

```
0000000 1 1
0000001 1 1
0000010 1 1
0000011 -1 -1
0000100 1 1
0000101 -1 -1
```

```
0000110 -1 -1
0000111 -1 -1
0001000 1 1
0001001 -1 -1
0001010 -1 -1
0001011 -1 -1
0001100 -1 -1
0001101 -1 -1
0001110 -1 -1
0001111 1 1
0010000 1 1
0010001 -1 -1
0010010 -1 -1
0010011 -1 -1
0010100 -1 -1
0010101 -1 -1
0010110 -1 -1
0010111 1 1
0011000 -1 -1
0011001 -1 -1
0011010 -1 -1
0011011 1 1
0011100 -1 -1
0011101 1 1
0011110 1 1
0011111 1 1
0100000 1 1
0100001 -1 -1
0100010 -1 -1
0100011 -1 -1
0100100 -1 -1
0100101 -1 -1
0100110 -1 -1
0100111 1 1
0101000 -1 -1
0101001 -1 -1
0101010 -1 -1
0101011 1 1
0101100 -1 -1
0101101 1 1
0101110 1 1
0101111 1 1
0110000 -1 -1
0110001 -1 -1
0110010 -1 -1
0110011 1 1
0110100 -1 -1
0110101 1 1
```

```
0110110 1 1
0110111 1 1
0111000 -1 -1
0111001 1 1
0111010 1 1
0111011 1 1
0111100 1 1
0111101 1 1
0111110 1 1
0111111 -1 -1
1000000 -1 -1
1000001 1 1
1000010 1 1
1000011 1 1
1000100 1 1
1000101 1 1
1000110 1 1
1000111 -1 -1
1001000 1 1
1001001 1 1
1001010 1 1
1001011 -1 -1
1001100 1 1
1001101 -1 -1
1001110 -1 -1
1001111 -1 -1
1010000 1 1
1010001 1 1
1010010 1 1
1010011 -1 -1
1010100 1 1
1010101 -1 -1
1010110 -1 -1
1010111 -1 -1
1011000 1 1
1011001 -1 -1
1011010 -1 -1
1011011 -1 -1
1011100 -1 -1
1011101 -1 -1
1011110 -1 -1
1011111 1 1
1100000 1 1
1100001 1 1
1100010 1 1
1100011 -1 -1
1100100 1 1
1100101 -1 -1
```

```
1100110 -1 -1
1100111 -1 -1
1101000 1 1
1101001 -1 -1
1101010 -1 -1
1101011 -1 -1
1101100 -1 -1
1101101 -1 -1
1101110 -1 -1
1101111 1 1
1110000 1 1
1110001 -1 -1
1110010 -1 -1
1110011 -1 -1
1110100 -1 -1
1110101 -1 -1
1110110 -1 -1
1110111 1 1
1111000 -1 -1
1111001 -1 -1
1111010 -1 -1
1111011 1 1
1111100 -1 -1
1111101 1 1
1111110 1 1
1111111 1 1
```

[6]:
```
# another alternative: reversion composed with "odd dimension negation"
odd_involute = [1]*128
reversion = [1]*128
odd_reversion = [1]*128
for n in range(128):
    m = Grade[ n & 0b01010101 ]
    if 1 == 1&m: odd_involute[n] *= -1
    if Grade[n] in [2,3,6,7]: reversion[n] *= -1
    odd_reversion[n] = odd_involute[n] * reversion[n]
# does not work
```

[7]:
```
# This is just a variation of A&S 7D inverse using a different A&D formula for␣
 ↪the determinant.

VERBOSE = True

def K(W): # just change the sign on scalar and pseudoscalar
    V = W.copy()
    V.Reg[0] *= -1
    V.Reg[127] *= -1
```

```
    return V

def L(W):
    return W * K( W * W )

def also_7d_inverse( U ):
    # Up = Signs236 * U # the originial, using e1 special after H&S
    Up = alt236 * U # the alternate involution template, using e7 instead of e1.
    W = U * Up
    #Wa = K( W )
    #Wb = K( Wa * K( Wa * Wa ) )
    #Wc = W * K( W * W )
    #I = Wc + Wb + Wb
    I = L(W) + 2*K(L(K(W)))
    V = I * W
    # IW = V so 1/W = I/V : because this reciprocal is what we care about
    # it does not matter if I and V contain the common factor -1/3
    # needed to express the determinant
    # V is all zero except for scalar and pseudoscalar parts
    Vp = GISigns * V # use the grade involution to obtain something like a␣
 ↪conjugate in the complex sense
    D = Vp * V # is now a scalar
    #util.printSparceAccum( D )
    d = D.Reg[0]
    # D = V'V so 1/V = V'/D
    # remember 1/W = I/V so 1/W = IV'/D
    rW = (1/d) * I * Vp # this is 1/W
    # remember W = UUp so 1/U = Up/W
    rU = Up * rW
    if VERBOSE:
        print('\nValidate that we have found rW = 1/W, by printing W * rW' )
        util.print_sparse( W * rW )
        print('\nValidate that we have found rU = 1/U, by printing U * rU' )
        util.print_sparse( U * rU )
    return rU

U = Accum.random()
rU = also_7d_inverse( U )
```

```
Validate that we have found rW = 1/W, by printing W * rW
  0. 00000000      1.00000000

Validate that we have found rU = 1/U, by printing U * rU
  0. 00000000      1.00000000
```

Let $[W]$ mean "just the scalar and pseudoscalar parts of $W$", then $b(W) = W - 2[W]$ negates those

parts.

$$b(W^2) = W^2 - 2[W^2]$$

So, it's fair to say that the expression above is *reduced*: we can accept our determinant algorithm in terms of $W^n$ and $[W^n]$. For instance:

$$(b(W))^2 = (W - 2[W])^2 = W^2 - 4W[W] + 4[W]$$

This strategy looks good since it tracks with the 6D work, and that means we can just write down the key ingredient for the determinant formula. Here is the $S = S(W)$ function satisfying: $-3det(W) = S * W$

$$S(W) = Wb(W^2) + 2b(b(W)b((b(W))^2))$$

in terms of $W^n$ and $[W^n]$ can be written:

$$S(W) = 3W^3 - 12[W]W^2 + 24[W]^2W - 6[W^2]W - 4[W^3] + 24[W][W^2] - 32[W]^3$$

Following the 6D example, we successively factor:

$$S_1 = (W - 4[W])W,$$

$$S_0 = (S_1 - 2[S_1])W$$

$$-\frac{1}{3}det(W) = (S_0 - \frac{4}{3}[S_0])W$$

[8]:
```
# This is the "tweak version" of the 7D inverse

VERBOSE = True

# we turn the negation function into a tweak function
def K( W, s=1 ):
    V = W.copy()
    V.Reg[0] *= -s
    V.Reg[127] *= -s
    return V

def another_7d_inverse( U ):
    #Up = Signs236 * U # as given by A&S using e1 after H&S
    Up = alt236 * U # the A&S reversion-like involution using e7 instead of e1␣
 ↪as special
    W = U * Up
    I1 = K(  W,  3 ) * W
    I0 = K( I1,  1 ) * W
    I  = K( I0, 1/3)
    V = I * W
    # IW = V so 1/W = I/V : because this reciprocal is what we care about
    # it does not matter if I and V contain the common factor -1/3
    # needed to express the determinant
    # V is all zero except for scalar and pseudoscalar parts
```

```
    Vp = GISigns * V # use the grade involution to obtain something like a␣
 ↪conjugate in the complex sense
    D = Vp * V # is now a scalar
    d = D.Reg[0]
    # D = V'V so 1/V = V'/D
    # remember 1/W = I/V so 1/W = IV'/D
    rW = (1/d) * I * Vp # this is 1/W
    # remember W = UUp so 1/U = Up/W
    rU = Up * rW
    if VERBOSE:
        print('\nValidate that we have found rW = 1/W, by printing W * rW' )
        util.print_sparse( W * rW )
        print('\nValidate that we have found rU = 1/U, by printing U * rU' )
        util.print_sparse( U * rU )
    return rU

import clif.context as Clif
Clif.Cl(7)
U = Accum.random()
rU = another_7d_inverse( U )
```

```
Validate that we have found rW = 1/W, by printing W * rW
  0. 00000000      1.00000000

Validate that we have found rU = 1/U, by printing U * rU
  0. 00000000      1.00000000
```

Here is an important version of 7D inverse. When we borrow the `Signs236` involution, we can use the next level of real tweaks. The *complex tweaks* in the A&S like version let us by with fewer multivector products instead of compound tweaks. That makes the other version likely to be faster, but it's special cased. This version is interesting in its amphibious nature between FLS and Jones.

One gets the impression that if the right extention of `Signs236` to 8D were known it would just work.

[9]:
```
import clif.context as Clif
Clif.Cl(7)
A = Accum.random()

#Z = Signs236
Z = alt236

def I7(A):
    '''returns the inverse of a 7D multivector'''
    V = Z * A
    B = A * V
    G = B.copy()
```

```
    G.Reg[0] *= -7
    G *= B
    G.Reg[0] *= -3
    G *= B
    G.Reg[0] *= -5/3
    G *= B
    G.Reg[0] *= -1
    G *= B
    G.Reg[0] *= -3/5
    G *= B
    G.Reg[0] *= -1/3
    G *= B
    G.Reg[0] *= -1/7
    I = G.copy()
    G *= B # G is now a scalar
    # print(G)
    # 1 = B(I/G) = A(~AI/G)
    return V*I.scale(1/G.Reg[0])

I = I7(A)
print(I*A)
```

```
00000000       1.00000000
```

[10]:
```
import numpy
import clif.context as Clif
from clif.multivector import Accum

TOLERANCE = 0.000000001

def near( a, b ):
    d = a - b
    verdict = -TOLERANCE < d and d < TOLERANCE
    return verdict

def verify_inverse( A, I ):
    R = A*I
    if not near( R.Reg[0], 1.0 ):
        return False
    for r in R.Reg[1:]:
        if not near( r, 0.0 ):
            return False
    return True

samples = numpy.random.normal( 0.0, 1.0, 1<<7 )
signatures = [(p, q, 7-p-q) for p in range(8) for q in range(8-p)]
```

```
VERBOSE = False
for s in signatures:
    Clif.Cl(*s) # initialize signature
    msg = f"Cl{s}: "
    for n in range(Clif.dimensions):
        A = Accum()
        A.Reg[1<<n] = 1.0 # e_n
        C = A*A
        c = C.Reg[0]
        msg += f"e{n+1}*e{n+1} = {int(c):2d}, "
    A.Reg = numpy.array(samples).tolist()
    I = another_7d_inverse( A )
#    print(I*A)
    if verify_inverse( A, I ):
        msg += "passed"
    else:
        msg += "failed"
    print( msg )
```

```
Cl(0, 0, 7): e1*e1 =  0, e2*e2 =  0, e3*e3 =  0, e4*e4 =  0, e5*e5 =  0, e6*e6 =
0, e7*e7 =  0, passed
Cl(0, 1, 6): e1*e1 = -1, e2*e2 =  0, e3*e3 =  0, e4*e4 =  0, e5*e5 =  0, e6*e6 =
0, e7*e7 =  0, passed
Cl(0, 2, 5): e1*e1 = -1, e2*e2 = -1, e3*e3 =  0, e4*e4 =  0, e5*e5 =  0, e6*e6 =
0, e7*e7 =  0, passed
Cl(0, 3, 4): e1*e1 = -1, e2*e2 = -1, e3*e3 = -1, e4*e4 =  0, e5*e5 =  0, e6*e6 =
0, e7*e7 =  0, passed
Cl(0, 4, 3): e1*e1 = -1, e2*e2 = -1, e3*e3 = -1, e4*e4 = -1, e5*e5 =  0, e6*e6 =
0, e7*e7 =  0, passed
Cl(0, 5, 2): e1*e1 = -1, e2*e2 = -1, e3*e3 = -1, e4*e4 = -1, e5*e5 = -1, e6*e6 =
0, e7*e7 =  0, passed
Cl(0, 6, 1): e1*e1 = -1, e2*e2 = -1, e3*e3 = -1, e4*e4 = -1, e5*e5 = -1, e6*e6 =
-1, e7*e7 =  0, passed
Cl(0, 7, 0): e1*e1 = -1, e2*e2 = -1, e3*e3 = -1, e4*e4 = -1, e5*e5 = -1, e6*e6 =
-1, e7*e7 = -1, passed
Cl(1, 0, 6): e1*e1 =  1, e2*e2 =  0, e3*e3 =  0, e4*e4 =  0, e5*e5 =  0, e6*e6 =
0, e7*e7 =  0, passed
Cl(1, 1, 5): e1*e1 =  1, e2*e2 = -1, e3*e3 =  0, e4*e4 =  0, e5*e5 =  0, e6*e6 =
0, e7*e7 =  0, passed
Cl(1, 2, 4): e1*e1 =  1, e2*e2 = -1, e3*e3 = -1, e4*e4 =  0, e5*e5 =  0, e6*e6 =
0, e7*e7 =  0, passed
Cl(1, 3, 3): e1*e1 =  1, e2*e2 = -1, e3*e3 = -1, e4*e4 = -1, e5*e5 =  0, e6*e6 =
0, e7*e7 =  0, passed
Cl(1, 4, 2): e1*e1 =  1, e2*e2 = -1, e3*e3 = -1, e4*e4 = -1, e5*e5 = -1, e6*e6 =
0, e7*e7 =  0, passed
Cl(1, 5, 1): e1*e1 =  1, e2*e2 = -1, e3*e3 = -1, e4*e4 = -1, e5*e5 = -1, e6*e6 =
-1, e7*e7 =  0, passed
Cl(1, 6, 0): e1*e1 =  1, e2*e2 = -1, e3*e3 = -1, e4*e4 = -1, e5*e5 = -1, e6*e6 =
```

```
-1, e7*e7 = -1, passed
Cl(2, 0, 5): e1*e1 =  1, e2*e2 =  1, e3*e3 =  0, e4*e4 =  0, e5*e5 =  0, e6*e6 =
0, e7*e7 =  0, passed
Cl(2, 1, 4): e1*e1 =  1, e2*e2 =  1, e3*e3 = -1, e4*e4 =  0, e5*e5 =  0, e6*e6 =
0, e7*e7 =  0, passed
Cl(2, 2, 3): e1*e1 =  1, e2*e2 =  1, e3*e3 = -1, e4*e4 = -1, e5*e5 =  0, e6*e6 =
0, e7*e7 =  0, passed
Cl(2, 3, 2): e1*e1 =  1, e2*e2 =  1, e3*e3 = -1, e4*e4 = -1, e5*e5 = -1, e6*e6 =
0, e7*e7 =  0, passed
Cl(2, 4, 1): e1*e1 =  1, e2*e2 =  1, e3*e3 = -1, e4*e4 = -1, e5*e5 = -1, e6*e6 =
-1, e7*e7 =  0, passed
Cl(2, 5, 0): e1*e1 =  1, e2*e2 =  1, e3*e3 = -1, e4*e4 = -1, e5*e5 = -1, e6*e6 =
-1, e7*e7 = -1, passed
Cl(3, 0, 4): e1*e1 =  1, e2*e2 =  1, e3*e3 =  1, e4*e4 =  0, e5*e5 =  0, e6*e6 =
0, e7*e7 =  0, passed
Cl(3, 1, 3): e1*e1 =  1, e2*e2 =  1, e3*e3 =  1, e4*e4 = -1, e5*e5 =  0, e6*e6 =
0, e7*e7 =  0, passed
Cl(3, 2, 2): e1*e1 =  1, e2*e2 =  1, e3*e3 =  1, e4*e4 = -1, e5*e5 = -1, e6*e6 =
0, e7*e7 =  0, passed
Cl(3, 3, 1): e1*e1 =  1, e2*e2 =  1, e3*e3 =  1, e4*e4 = -1, e5*e5 = -1, e6*e6 =
-1, e7*e7 =  0, passed
Cl(3, 4, 0): e1*e1 =  1, e2*e2 =  1, e3*e3 =  1, e4*e4 = -1, e5*e5 = -1, e6*e6 =
-1, e7*e7 = -1, passed
Cl(4, 0, 3): e1*e1 =  1, e2*e2 =  1, e3*e3 =  1, e4*e4 =  1, e5*e5 =  0, e6*e6 =
0, e7*e7 =  0, passed
Cl(4, 1, 2): e1*e1 =  1, e2*e2 =  1, e3*e3 =  1, e4*e4 =  1, e5*e5 = -1, e6*e6 =
0, e7*e7 =  0, passed
Cl(4, 2, 1): e1*e1 =  1, e2*e2 =  1, e3*e3 =  1, e4*e4 =  1, e5*e5 = -1, e6*e6 =
-1, e7*e7 =  0, passed
Cl(4, 3, 0): e1*e1 =  1, e2*e2 =  1, e3*e3 =  1, e4*e4 =  1, e5*e5 = -1, e6*e6 =
-1, e7*e7 = -1, passed
Cl(5, 0, 2): e1*e1 =  1, e2*e2 =  1, e3*e3 =  1, e4*e4 =  1, e5*e5 =  1, e6*e6 =
0, e7*e7 =  0, passed
Cl(5, 1, 1): e1*e1 =  1, e2*e2 =  1, e3*e3 =  1, e4*e4 =  1, e5*e5 =  1, e6*e6 =
-1, e7*e7 =  0, passed
Cl(5, 2, 0): e1*e1 =  1, e2*e2 =  1, e3*e3 =  1, e4*e4 =  1, e5*e5 =  1, e6*e6 =
-1, e7*e7 = -1, passed
Cl(6, 0, 1): e1*e1 =  1, e2*e2 =  1, e3*e3 =  1, e4*e4 =  1, e5*e5 =  1, e6*e6 =
1, e7*e7 =  0, passed
Cl(6, 1, 0): e1*e1 =  1, e2*e2 =  1, e3*e3 =  1, e4*e4 =  1, e5*e5 =  1, e6*e6 =
1, e7*e7 = -1, passed
Cl(7, 0, 0): e1*e1 =  1, e2*e2 =  1, e3*e3 =  1, e4*e4 =  1, e5*e5 =  1, e6*e6 =
1, e7*e7 =  1, passed
```

[11]:
```
import clif.context as Clif
from clif.multivector import Accum
import I7_instrumented as I7I
```

```
import clif.util as util

def show_clifford_environment():
    msg = ""
    for n in range(Clif.dimensions):
        A = Accum()
        A.Reg[1<<n] = 1.0 # e_n
        C = A*A
        c = C.Reg[0]
        msg += f"e{n+1}*e{n+1} = {int(c):2d}, "
    return msg

Clif.Cl(7) # initialize signature

Areg = [0.0]*128
Areg[0b0000000] =  2.0 # scalar
Areg[0b0000001] = -2.0 # e1
Areg[0b0100000] =  1.0 # e6
Areg[0b0010010] = -3.0 # e25
Areg[0b0010101] = -1.0 # e135
Areg[0b0111100] =  1.0 # e3456
Areg[0b1110110] = -2.0 # e23567
Areg[0b1111011] =  1.0 # e124567
Areg[0b1111111] =  3.0 # e1234567

divider =␣
 ↪"\n============================================================================="

print( divider, "\nTEST #1: Euclidean" )
Clif.Cl(7)
print( "Cl(7): ", show_clifford_environment() )
A = Accum()
A.Reg = Areg
iA = I7I.I7_instrumented( A )

print( divider, "\nTEST #2: Anti-Euclidean" )
Clif.Cl(0,7)
print( "Cl(0,7): ", show_clifford_environment() )
A = Accum()
A.Reg = Areg
iA = I7I.I7_instrumented( A )

print( divider, "\nTEST #3: Grassman" )
Clif.Cl(0,0,7)
print( "Cl(0,0,7): ", show_clifford_environment() )
A = Accum()
A.Reg = Areg
```

```
iA = I7I.I7_instrumented( A )

print( divider, "\nTEST #4: Mixed" )
Clif.Cl(2,3,2)
print( "Cl(2,3,2): ", show_clifford_environment() )
A = Accum()
A.Reg = Areg
iA = I7I.I7_instrumented( A )

print( divider, "\nTEST #5: Conformal" )
Clif.Cl(6,0,1)
print( "Cl(6,0,1): ", show_clifford_environment() )
A = Accum()
A.Reg = Areg
iA = I7I.I7_instrumented( A )
```

```
================================================================================
TEST #1: Euclidean
Cl(7):  e1*e1 =  1, e2*e2 =  1, e3*e3 =  1, e4*e4 =  1, e5*e5 =  1, e6*e6 =  1,
e7*e7 =  1,

U =  2 - (2^e1) - (3^e25) - (1^e135) + (1^e6) + (1^e3456) - (2^e23567) +
(1^e124567) + (3^e1234567)

Up = Signs236 * U =  2 + (2^e1) + (3^e25) - (1^e135) + (1^e6) + (1^e3456) -
(2^e23567) + (1^e124567) + (3^e1234567)

W = U * Up =  4 - (6^e3) + (6^e123) - (12^e14) + (4^e134) - (12^e125) - (4^e135)
+ (4^e6) + (4^e16) - (2^e146) + (6^e2346) + (4^e3456) + (6^e127) + (4^e2357) -
(6^e123457) - (6^e2467) + (2^e23467) - (8^e23567) - (8^e123567) + (4^e24567) +
(4^e124567) + (12^e1234567)

I1 = K(  W,  3 ) * W =  96 - (72^e12) + (72^e3) + (96^e13) - (24^e123) +
(176^e4) + (272^e14) - (176^e134) - (48^e5) - (48^e15) + (96^e125) - (160^e135)
+ (32^e145) - (96^e2345) + (128^e6) + (32^e16) - (48^e126) - (112^e146) -
(72^e2346) - (56^e156) - (48^e2456) + (48^e3456) + (24^e7) - (136^e127) -
(96^e137) + (32^e2347) + (96^e157) - (224^e2357) + (64^e23457) + (16^e123457) +
(176^e2467) - (288^e3467) - (56^e23467) + (32^e123467) - (96^e2567) -
(200^e23567) + (64^e123567) + (192^e4567) + (32^e24567) + (160^e124567) -
(48^e34567) - (80^e1234567)

I0 = K( I1,  1 ) * W =  3216 + (1152^e2) + (1008^e12) + (992^e3) + (2016^e123) +
(576^e4) - (1056^e14) + (1504^e134) - (384^e5) - (640^e15) - (1824^e125) -
(864^e135) + (448^e145) + (2304^e2345) - (2016^e6) - (1184^e16) + (576^e126) -
(224^e136) - (2176^e146) + (288^e2346) + (1616^e156) - (336^e2356) -
(1248^e2456) + (2016^e3456) - (192^e123456) - (336^e7) - (928^e127) +
(2688^e137) - (192^e147) - (32^e2347) + (1824^e157) + (1440^e2357) + (128^e2457)
```

```
- (1008^e3457) - (512^e23457) + (1152^e123457) - (1056^e167) + (352^e2367) -
(2544^e2467) - (960^e3467) + (1200^e23467) + (1088^e123467) - (1984^e2567) -
(1952^e23567) - (1088^e123567) - (288^e4567) - (608^e24567) - (1216^e124567) -
(576^e34567) - (960^e134567) + (3936^e1234567)

I  = K( I0, 1/3) =  -1072 + (1152^e2) + (1008^e12) + (992^e3) + (2016^e123) +
(576^e4) - (1056^e14) + (1504^e134) - (384^e5) - (640^e15) - (1824^e125) -
(864^e135) + (448^e145) + (2304^e2345) - (2016^e6) - (1184^e16) + (576^e126) -
(224^e136) - (2176^e146) + (288^e2346) + (1616^e156) - (336^e2356) -
(1248^e2456) + (2016^e3456) - (192^e123456) - (336^e7) - (928^e127) +
(2688^e137) - (192^e147) - (32^e2347) + (1824^e157) + (1440^e2357) + (128^e2457)
- (1008^e3457) - (512^e23457) + (1152^e123457) - (1056^e167) + (352^e2367) -
(2544^e2467) - (960^e3467) + (1200^e23467) + (1088^e123467) - (1984^e2567) -
(1952^e23567) - (1088^e123567) - (288^e4567) - (608^e24567) - (1216^e124567) -
(576^e34567) - (960^e134567) - (1312^e1234567)

V = I * W =  -3264 + (7680^e1234567)

Vp = GISigns * V =  -3264 - (7680^e1234567)

D = Vp * V =  6.96361e+07

rW = (1/d) * I * Vp = 1/W =  -0.0944503 + (0.0518793^e2) + (0.0162785^e12) -
(0.180607^e3) - (0.0670549^e13) - (0.0627316^e123) + (0.0929945^e4) +
(0.264779^e14) + (0.148315^e134) + (0.137992^e5) + (0.162343^e15) +
(0.191371^e125) - (0.240074^e135) - (0.05982^e145) - (0.224457^e2345) -
(0.0325569^e6) + (0.111964^e16) - (0.138168^e126) - (0.00361743^e136) +
(0.260808^e146) - (0.214664^e2346) - (0.0722163^e156) - (0.00542615^e2356) -
(0.237957^e2456) - (0.196841^e3456) + (0.0460561^e123456) - (0.00542615^e7) -
(0.178842^e127) - (0.263632^e137) + (0.0460561^e147) + (0.179725^e2347) -
(0.0537321^e157) + (0.17249^e2357) - (0.0307041^e2457) - (0.0162785^e3457) -
(0.106582^e23457) - (0.276337^e123457) - (0.204606^e167) + (0.0329098^e2367) +
(0.214531^e2467) - (0.156167^e3467) + (0.0143374^e23467) - (0.00864655^e123467)
+ (0.258867^e2567) - (0.0249691^e23567) + (0.114523^e123567) + (0.235839^e4567)
+ (0.0284983^e24567) - (0.0524087^e124567) + (0.138168^e34567) +
(0.172049^e134567) + (0.179725^e1234567)

W * rW =  1

rU = Up * rW = 1/U =  -1.0952 - (0.908594^e1) + (0.173372^e2) + (0.105788^e3) -
(0.0888477^e14) - (0.270911^e24) - (0.2991^e124) + (0.972296^e34) +
(1.0656^e134) + (0.257014^e234) + (0.0162343^e15) - (0.052541^e25) -
(0.0857597^e125) - (0.488795^e35) - (0.565952^e135) + (0.552673^e235) -
(0.794336^e1245) - (0.172049^e1345) + (0.164902^e2345) + (0.180254^e12345) -
(0.867522^e6) + (0.0284542^e1236) - (0.312423^e46) - (0.326981^e146) -
(0.0402329^e246) + (0.1994^e346) - (0.00282336^e56) - (0.00573496^e156) -
(0.143727^e256) + (0.0189695^e356) - (0.126875^e1456) - (1.26681^e2456) -
(1.53031^e12456) - (0.334789^e3456) - (0.393683^e13456) - (0.00105876^e23456) +
```

```
(0.14955^e17) - (0.357685^e27) - (0.434798^e127) + (0.21837^e37) +
(0.186607^e137) - (0.00441151^e237) - (0.221105^e1247) + (0.823584^e1347) +
(0.351729^e2347) + (0.3988^e12347) - (0.00141168^e1257) - (0.067099^e1357) +
(1.48063^e2357) + (1.24528^e12357) + (0.331657^e457) + (0.100053^e123457) -
(1.16411^e67) - (1.08258^e167) + (0.0518793^e267) - (0.0749074^e367) -
(0.0651138^e1467) + (0.24678^e2467) + (0.291071^e12467) - (0.886448^e3467) -
(1.14108^e13467) + (0.126434^e23467) - (0.0717311^e1567) - (0.20337^e2567) -
(0.170108^e12567) + (0.00211752^e3567) - (0.078613^e13567) - (0.908594^e23567) -
(0.515793^e124567) - (0.113817^e134567) - (0.255514^e234567) -
(0.32204^e1234567)

U * rU =  1

================================================================================
TEST #2: Anti-Euclidean
Cl(0,7):  e1*e1 = -1, e2*e2 = -1, e3*e3 = -1, e4*e4 = -1, e5*e5 = -1, e6*e6 =
-1, e7*e7 = -1,

U =  2 - (2^e1) - (3^e25) - (1^e135) + (1^e6) + (1^e3456) - (2^e23567) +
(1^e124567) + (3^e1234567)

Up = Signs236 * U =  2 + (2^e1) + (3^e25) - (1^e135) + (1^e6) + (1^e3456) -
(2^e23567) + (1^e124567) + (3^e1234567)

W = U * Up =  22 - (6^e3) - (6^e123) + (12^e14) + (4^e134) - (12^e125) -
(4^e135) + (4^e6) + (4^e16) - (2^e146) - (6^e2346) + (4^e3456) + (6^e127) -
(4^e2357) + (6^e123457) + (6^e2467) + (2^e23467) - (8^e23567) - (8^e123567) -
(4^e24567) + (4^e124567) + (12^e1234567)

I1 = K(  W,  3 ) * W =  -1940 - (72^e12) + (432^e3) + (96^e13) + (240^e123) +
(176^e4) - (800^e14) + (96^e134) - (48^e5) + (48^e15) + (528^e125) + (368^e135)
+ (32^e145) + (96^e2345) - (336^e6) - (240^e16) + (48^e126) + (8^e146) +
(240^e2346) - (8^e156) + (48^e2456) - (352^e3456) - (24^e7) - (368^e127) +
(96^e137) - (32^e2347) - (192^e157) - (48^e2357) + (64^e23457) - (424^e123457) -
(392^e2467) + (288^e3467) + (16^e23467) + (32^e123467) + (96^e2567) +
(680^e23567) + (416^e123567) + (192^e4567) + (240^e24567) - (368^e124567) +
(48^e34567) - (1792^e1234567)

I0 = K( I1,  1 ) * W =  76296 - (1152^e2) + (2016^e12) - (6104^e3) - (2496^e13)
- (3672^e123) - (5216^e4) + (13328^e14) - (6544^e134) + (672^e5) - (2656^e15) -
(9072^e125) - (6416^e135) - (256^e145) + (1344^e2345) + (6736^e6) + (2640^e16) +
(96^e126) - (32^e136) - (2136^e146) - (4440^e2346) - (480^e156) - (48^e2356) -
(192^e2456) + (6896^e3456) - (384^e123456) + (288^e7) + (7064^e127) + (576^e137)
- (192^e147) - (480^e2347) + (3936^e157) + (2736^e2357) - (128^e2457) +
(240^e3457) - (2688^e23457) + (6664^e123457) + (1056^e167) - (352^e2367) +
(6584^e2467) - (8448^e3467) - (3016^e23467) - (768^e123467) - (6016^e2567) -
(13232^e23567) - (5408^e123567) - (3360^e4567) - (2640^e24567) + (5936^e124567)
- (1824^e34567) - (1536^e134567) + (74064^e1234567)
```

```
I  = K( I0, 1/3) =  -25432 - (1152^e2) + (2016^e12) - (6104^e3) - (2496^e13) -
(3672^e123) - (5216^e4) + (13328^e14) - (6544^e134) + (672^e5) - (2656^e15) -
(9072^e125) - (6416^e135) - (256^e145) + (1344^e2345) + (6736^e6) + (2640^e16) +
(96^e126) - (32^e136) - (2136^e146) - (4440^e2346) - (480^e156) - (48^e2356) -
(192^e2456) + (6896^e3456) - (384^e123456) + (288^e7) + (7064^e127) + (576^e137)
- (192^e147) - (480^e2347) + (3936^e157) + (2736^e2357) - (128^e2457) +
(240^e3457) - (2688^e23457) + (6664^e123457) + (1056^e167) - (352^e2367) +
(6584^e2467) - (8448^e3467) - (3016^e23467) - (768^e123467) - (6016^e2567) -
(13232^e23567) - (5408^e123567) - (3360^e4567) - (2640^e24567) + (5936^e124567)
- (1824^e34567) - (1536^e134567) - (24688^e1234567)

V = I * W =  -1.04738e+06 - (1.0455e+06^e1234567)

Vp = GISigns * V =  -1.04738e+06 + (1.0455e+06^e1234567)

D = Vp * V =  3.91787e+09

rW = (1/d) * I * Vp = 1/W =  0.210692 - (0.101922^e2) - (0.0521994^e12) +
(0.0477482^e3) - (0.0372345^e13) + (0.0850133^e123) - (0.0487439^e4) -
(0.0319863^e14) + (0.144026^e134) + (0.025297^e5) - (0.0947988^e15) +
(0.170852^e125) - (0.041766^e135) - (0.0254958^e145) - (0.0774964^e2345) -
(0.0224321^e6) + (0.0115476^e16) - (0.0897092^e126) - (0.0256028^e136) -
(0.159092^e146) + (0.136616^e2346) + (0.000229349^e156) - (0.0384042^e2356) -
(0.102381^e2456) + (0.0415367^e3456) + (0.0258016^e123456) + (0.0254805^e7) -
(0.0482069^e127) - (0.102748^e137) + (0.0385189^e147) + (0.000229349^e2347) +
(0.132614^e157) - (0.16142^e2357) + (0.0256793^e2457) - (0.089778^e3457) +
(0.0140934^e23457) + (0.0160294^e123457) + (0.0763497^e167) + (0.0257863^e2367)
- (0.0479776^e2467) - (0.162481^e3467) + (0.0975089^e23467) + (0.025985^e123467)
- (0.138025^e2567) - (0.0192957^e23567) + (0.0538202^e123567) -
(0.0816534^e4567) + (0.0396886^e24567) + (0.0419954^e124567) -
(0.0503647^e34567) + (0.103206^e134567) - (0.186744^e1234567)

W * rW =  1

rU = Up * rW = 1/U =  -0.1137 + (0.0446082^e1) + (0.0772671^e2) - (0.0994659^e3)
+ (0.0150317^e14) + (0.0623251^e24) - (0.0524288^e124) + (0.10313^e34) -
(0.0229653^e134) + (0.0266902^e234) + (0.112978^e15) + (0.0687413^e25) -
(0.145739^e125) + (0.057547^e35) + (0.0431652^e135) - (0.052641^e235) +
(0.095959^e1245) + (0.0345053^e1345) + (0.0740562^e2345) + (0.161575^e12345) -
(0.0858295^e6) - (0.0904373^e1236) - (0.00951406^e46) + (0.082846^e146) +
(0.0211401^e246) + (0.0260615^e346) + (0.0504411^e56) + (0.0314492^e156) -
(0.0287774^e256) - (0.0602915^e356) + (0.0117387^e1456) + (0.103481^e2456) +
(0.0763669^e12456) - (0.101612^e3456) + (0.0830734^e13456) - (0.0250677^e23456)
+ (0.0262144^e17) + (0.0964138^e27) + (0.0778748^e127) - (0.0902998^e37) +
(0.101647^e137) - (0.0139023^e237) + (0.0679136^e1247) - (0.00968412^e1347) -
(0.0326533^e2347) + (0.052123^e12347) + (0.0252206^e1257) - (0.0173214^e1357) +
(0.0838207^e2357) + (0.016127^e12357) + (0.10187^e457) + (0.0680167^e123457) +
```

```
(0.146117^e67) - (0.0797899^e167) + (0.0424178^e267) - (0.0578871^e367) +
(0.0371026^e1467) + (0.0209374^e2467) - (0.0706581^e12467) + (0.187595^e3467) +
(0.0210023^e13467) - (0.10497^e23467) + (0.0374123^e1567) + (0.0283168^e2567) +
(0.101998^e12567) - (0.0501353^e3567) - (0.06588^e13567) + (0.0490709^e23567) -
(0.118483^e124567) - (0.0765791^e134567) + (0.00667386^e234567) +
(0.258095^e1234567)

U * rU =  1

======================================================================
TEST #3: Grassman
Cl(0,0,7):  e1*e1 =  0, e2*e2 =  0, e3*e3 =  0, e4*e4 =  0, e5*e5 =  0, e6*e6 =
0, e7*e7 =  0,

U =  2 - (2^e1) - (3^e25) - (1^e135) + (1^e6) + (1^e3456) - (2^e23567) +
(1^e124567) + (3^e1234567)

Up = Signs236 * U =  2 + (2^e1) + (3^e25) - (1^e135) + (1^e6) + (1^e3456) -
(2^e23567) + (1^e124567) + (3^e1234567)

W = U * Up =  4 - (12^e125) - (4^e135) + (4^e6) + (4^e16) + (4^e3456) -
(8^e23567) - (8^e123567) + (4^e124567) + (12^e1234567)

I1 = K(  W,  3 ) * W =  -48 + (96^e125) + (32^e135) - (32^e6) - (32^e16) -
(32^e3456) + (64^e23567) + (64^e123567) - (32^e124567) - (288^e1234567)

I0 = K( I1,  1 ) * W =  192 - (192^e125) - (64^e135) + (64^e6) + (64^e16) +
(64^e3456) - (128^e23567) - (128^e123567) + (64^e124567) + (1728^e1234567)

I  = K( I0, 1/3) =  -64 - (192^e125) - (64^e135) + (64^e6) + (64^e16) +
(64^e3456) - (128^e23567) - (128^e123567) + (64^e124567) - (576^e1234567)

V = I * W =  -256 - (3072^e1234567)

Vp = GISigns * V =  -256 + (3072^e1234567)

D = Vp * V =  65536

rW = (1/d) * I * Vp = 1/W =  0.25 + (0.75^e125) + (0.25^e135) - (0.25^e6) -
(0.25^e16) - (0.25^e3456) + (0.5^e23567) + (0.5^e123567) - (0.25^e124567) -
(0.75^e1234567)

W * rW =  1

rU = Up * rW = 1/U =  0.5 + (0.5^e1) + (0.75^e25) + (1.5^e125) + (0.25^e135) -
(0.25^e6) - (0.75^e256) - (0.25^e3456) - (0.5^e13456) + (0.5^e23567) -
(0.25^e124567) - (0.75^e1234567)
```

```
U * rU =  1

================================================================================
TEST #4: Mixed
Cl(2,3,2):  e1*e1 =  1, e2*e2 =  1, e3*e3 = -1, e4*e4 = -1, e5*e5 = -1, e6*e6 =
0, e7*e7 =  0,

U =  2 - (2^e1) - (3^e25) - (1^e135) + (1^e6) + (1^e3456) - (2^e23567) +
(1^e124567) + (3^e1234567)

Up = Signs236 * U =  2 + (2^e1) + (3^e25) - (1^e135) + (1^e6) + (1^e3456) -
(2^e23567) + (1^e124567) + (3^e1234567)

W = U * Up =  -10 - (6^e123) - (12^e125) - (4^e135) + (4^e6) + (4^e16) -
(2^e146) - (6^e2346) + (4^e3456) - (6^e2467) - (2^e23467) - (8^e23567) -
(8^e123567) + (4^e24567) + (4^e124567) + (12^e1234567)

I1 = K(  W,  3 ) * W =  -136 - (120^e123) - (240^e125) - (80^e135) + (80^e6) +
(80^e16) - (96^e2346) - (48^e2456) + (64^e3456) - (24^e2467) + (288^e3467) -
(72^e23467) - (32^e123467) - (160^e23567) - (160^e123567) + (144^e4567) +
(80^e24567) + (64^e124567) + (48^e34567) + (672^e1234567)

I0 = K( I1,  1 ) * W =  1920 + (384^e123) + (768^e125) + (256^e135) - (256^e6) -
(256^e16) - (128^e146) + (384^e2346) - (256^e3456) + (1152^e2467) + (2304^e3467)
- (128^e23467) + (256^e123467) + (512^e23567) + (512^e123567) + (1152^e4567) -
(384^e24567) - (256^e124567) - (384^e134567) + (3456^e1234567)

I  = K( I0, 1/3) =  -640 + (384^e123) + (768^e125) + (256^e135) - (256^e6) -
(256^e16) - (128^e146) + (384^e2346) - (256^e3456) + (1152^e2467) + (2304^e3467)
- (128^e23467) + (256^e123467) + (512^e23567) + (512^e123567) + (1152^e4567) -
(384^e24567) - (256^e124567) - (384^e134567) - (1152^e1234567)

V = I * W =  -4096 - (24576^e1234567)

Vp = GISigns * V =  -4096 + (24576^e1234567)

D = Vp * V =  1.67772e+07

rW = (1/d) * I * Vp = 1/W =  0.15625 - (0.09375^e123) - (0.1875^e125) -
(0.0625^e135) + (0.0625^e6) + (0.0625^e16) + (0.03125^e146) - (0.09375^e2346) +
(0.0625^e3456) + (0.09375^e2467) + (0.5625^e3467) + (0.03125^e23467) -
(0.0625^e123467) - (0.125^e23567) - (0.125^e123567) + (0.28125^e4567) +
(0.09375^e24567) + (0.0625^e124567) + (0.09375^e134567) - (0.65625^e1234567)

W * rW =  1

rU = Up * rW = 1/U =  0.25 - (0.25^e1) - (0.375^e125) - (0.125^e35) +
(0.15625^e6) - (0.09375^e1236) + (0.0625^e46) - (0.0625^e146) + (0.1875^e256) +
```

```
(0.0625^e356) + (0.09375^e12456) + (0.03125^e3456) + (0.125^e13456) +
(0.125^e267) + (0.375^e367) - (0.1875^e1467) - (0.1875^e2467) + (0.1875^e12467)
+ (0.5625^e3467) - (0.5625^e13467) + (0.0625^e23467) + (0.1875^e1567) -
(0.3125^e23567) + (0.125^e124567) + (0.375^e234567) - (0.75^e1234567)

U * rU =  1

================================================================================
TEST #5: Conformal
Cl(6,0,1):  e1*e1 =  1, e2*e2 =  1, e3*e3 =  1, e4*e4 =  1, e5*e5 =  1, e6*e6 =
1, e7*e7 =  0,

U =  2 - (2^e1) - (3^e25) - (1^e135) + (1^e6) + (1^e3456) - (2^e23567) +
(1^e124567) + (3^e1234567)

Up = Signs236 * U =  2 + (2^e1) + (3^e25) - (1^e135) + (1^e6) + (1^e3456) -
(2^e23567) + (1^e124567) + (3^e1234567)

W = U * Up =  10 + (6^e123) - (12^e125) - (4^e135) + (4^e6) + (4^e16) - (2^e146)
+ (6^e2346) + (4^e3456) + (6^e127) + (4^e2357) - (6^e123457) - (6^e2467) +
(2^e23467) - (8^e23567) - (8^e123567) + (4^e24567) + (4^e124567) + (12^e1234567)

I1 = K(  W,  3 ) * W =  -448 - (96^e123) - (16^e4) - (16^e14) + (240^e125) +
(64^e135) - (80^e6) - (80^e16) - (64^e146) - (144^e2346) - (48^e2456) -
(96^e3456) + (24^e7) - (208^e127) - (96^e137) - (16^e2347) + (96^e157) -
(128^e2357) + (64^e23457) + (184^e123457) + (224^e2467) - (288^e3467) -
(8^e23467) + (32^e123467) + (152^e23567) + (128^e123567) + (192^e4567) -
(80^e24567) - (96^e124567) - (48^e34567) - (656^e1234567)

I0 = K( I1,  1 ) * W =  6816 + (1824^e123) - (256^e4) - (256^e14) - (2880^e125)
- (1216^e135) + (960^e6) + (960^e16) + (1120^e146) + (1056^e2346) - (768^e2456)
+ (704^e3456) - (480^e7) + (1376^e127) + (1536^e137) + (96^e147) + (160^e2347) -
(480^e157) + (1504^e2357) - (64^e2457) + (192^e3457) - (1344^e23457) -
(2784^e123457) + (96^e167) - (32^e2367) - (3168^e2467) + (2496^e3467) +
(384^e23467) + (384^e123467) + (64^e2567) - (1024^e23567) - (1408^e123567) -
(2016^e4567) + (864^e24567) + (704^e124567) + (960^e34567) + (288^e134567) +
(14880^e1234567)

I  = K( I0, 1/3) =  -2272 + (1824^e123) - (256^e4) - (256^e14) - (2880^e125) -
(1216^e135) + (960^e6) + (960^e16) + (1120^e146) + (1056^e2346) - (768^e2456) +
(704^e3456) - (480^e7) + (1376^e127) + (1536^e137) + (96^e147) + (160^e2347) -
(480^e157) + (1504^e2357) - (64^e2457) + (192^e3457) - (1344^e23457) -
(2784^e123457) + (96^e167) - (32^e2367) - (3168^e2467) + (2496^e3467) +
(384^e23467) + (384^e123467) + (64^e2567) - (1024^e23567) - (1408^e123567) -
(2016^e4567) + (864^e24567) + (704^e124567) + (960^e34567) + (288^e134567) -
(4960^e1234567)

V = I * W =  -61696 - (116224^e1234567)
```

```
Vp = GISigns * V =  -61696 + (116224^e1234567)

D = Vp * V =  3.8064e+09

rW = (1/d) * I * Vp = 1/W =  0.0368257 - (0.0295643^e123) + (0.00414938^e4) +
(0.00414938^e14) + (0.0466805^e125) + (0.0197095^e135) - (0.0155602^e6) -
(0.0155602^e16) - (0.0181535^e146) - (0.0171162^e2346) + (0.0124481^e2456) -
(0.0114108^e3456) + (0.00778008^e7) - (0.000807063^e127) - (0.00144626^e137) -
(0.00155602^e147) - (0.00259336^e2347) - (0.0244637^e157) + (0.00982034^e2357) +
(0.00103734^e2457) - (0.00311203^e3457) - (0.00752828^e23457) +
(0.015812^e123457) - (0.00155602^e167) + (0.000518672^e2367) + (0.0142194^e2467)
+ (0.0474811^e3467) - (0.00622407^e23467) - (0.00622407^e123467) -
(0.00103734^e2567) + (0.0244142^e23567) + (0.0306382^e123567) -
(0.0230174^e4567) - (0.0140041^e24567) - (0.0114108^e124567) -
(0.0155602^e34567) - (0.00466805^e134567) + (0.0110212^e1234567)

W * rW =  1

rU = Up * rW = 1/U =  0.06639 - (0.0819502^e1) - (0.0181535^e14) -
(0.0171162^e234) + (0.216286^e25) + (0.093361^e125) + (0.0394191^e35) +
(0.0731328^e135) - (0.0124481^e1245) + (0.0114108^e1345) + (0.0368257^e6) -
(0.0295643^e1236) + (0.00518672^e46) - (0.00103734^e146) - (0.0466805^e256) -
(0.0197095^e356) + (0.0248963^e2456) + (0.0326763^e12456) - (0.0554979^e3456) -
(0.0228216^e13456) - (0.00622407^e17) - (0.00161413^e27) + (0.0583861^e127) -
(0.0209126^e37) - (0.0402369^e137) - (0.0145228^e237) - (0.0381665^e1247) -
(0.0474359^e1347) + (0.0114108^e2347) - (0.00622407^e1257) + (0.0248963^e1357) -
(0.075328^e2357) - (0.0377791^e12357) + (0.0572498^e457) + (0.0460327^e123457) -
(0.00466805^e67) - (0.00311203^e167) - (0.02311^e267) + (0.0732425^e367) -
(0.0482365^e1467) + (0.0173142^e2467) + (0.0284387^e12467) - (0.0450793^e3467) +
(0.0604522^e13467) + (0.0181535^e23467) - (0.034665^e1567) + (0.00466805^e2567)
- (0.00311203^e12567) + (0.00155602^e13567) - (0.108368^e23567) +
(0.0482365^e124567) + (0.00311203^e134567) + (0.163715^e234567) +
(0.121821^e1234567)

U * rU =  1
```